

Application API Design

1. Architecture Overview

◆ Architecture Pattern

Use:

Controller → Service → Repository → Database

But with clean layering:

API Layer (Controllers)

Business Layer (Services)

Domain Layer (Entities + DTOs)

Data Layer (Repositories)

Security Layer (JWT + Filters)

2. Package Structure (Production Level)

com.yourapp.notemanagement

```
|— config/           # App configs (JWT, CORS, Security)
|— controller/       # REST Controllers
|— service/          # Business logic
|— repository/       # JPA repositories
|— entity/           # Database entities
|— dto/              # Request / Response DTOs
|— mapper/           # MapStruct mappers
|— security/         # JWT auth system
|— exception/        # Global exception handling
|— util/             # Helpers (encryption, etc.)
|— ai/               # AI integration services
|— version/          # Version system logic
|— activity/         # Activity tracking
```

3. Authentication & Security API

◆ Auth Endpoints

POST /api/auth/register
POST /api/auth/login
POST /api/auth/refresh
GET /api/auth/me

◆ JWT Flow

- Login → Access Token + Refresh Token
 - Secure endpoints with JWT filter
 - Store roles in token
-

◆ Password Encryption

- Use BCrypt
 - Per-item lock password also encrypted
-

4. User & Friends API

GET /api/users/me
PUT /api/users/update
GET /api/users/search?q=
POST /api/friends/add
GET /api/friends/list
DELETE /api/friends/{id}

5. Group API (Core System)

◆ Group CRUD

POST /api/groups
GET /api/groups
GET /api/groups/{id}
PUT /api/groups/{id}
DELETE /api/groups/{id}

◆ Advanced Group Features

POST /api/groups/{id}/move
POST /api/groups/{id}/duplicate
POST /api/groups/{id}/archive
POST /api/groups/{id}/favorite
POST /api/groups/{id}/lock
POST /api/groups/{id}/unlock
POST /api/groups/{id}/share

◆ Nested Groups

GET /api/groups/tree

Return full hierarchy:

```
{
  "id": 1,
  "name": "Work",
  "children": [
    {
      "id": 2,
      "name": "Project A"
    }
  ]
}
```

6. Document API (Main Feature)

◆ Document CRUD

POST /api/docs
GET /api/docs
GET /api/docs/{id}

PUT /api/docs/{id}
DELETE /api/docs/{id}

◆ **Advanced Document Actions**

POST /api/docs/{id}/move
POST /api/docs/{id}/duplicate
POST /api/docs/{id}/copy
POST /api/docs/{id}/archive
POST /api/docs/{id}/favorite
POST /api/docs/{id}/lock
POST /api/docs/{id}/unlock
POST /api/docs/{id}/reminder

◆ **Search API**

GET /api/docs/search?q=

7. AI API

POST /api/ai/summarize
POST /api/ai/tags
POST /api/ai/category
POST /api/ai/rewrite

◆ **Example Request**

```
{  
  "content": "long note text..."  
}
```

◆ **Response**

```
{  
  "summary": "Short AI summary..."  
}
```

8. Version Control API

◆ Version Endpoints

POST /api/versions/{docId}
GET /api/versions/{docId}
GET /api/versions/{id}
POST /api/versions/{id}/restore

◆ Version Entity

```
Version {  
  id  
  documentId  
  label  
  content  
  createdAt  
  createdBy  
}
```

9. Collaboration API

POST /api/share/{docId}
GET /api/share/{docId}
PUT /api/share/{docId}/{userId}
DELETE /api/share/{docId}/{userId}

◆ Roles

- OWNER
 - EDITOR
 - VIEWER
-

10. Bin / Trash API

GET /api/bin
POST /api/bin/{id}/restore
DELETE /api/bin/{id}/permanent

11. Activity Log API

GET /api/activities

♦ Track Events

- CREATE_DOC
 - UPDATE_DOC
 - DELETE_DOC
 - SHARE
 - VERSION_CREATE
-

12. Reminder API

POST /api/reminders

GET /api/reminders

DELETE /api/reminders/{id}

13. Export API

GET /api/docs/{id}/export?type=pdf

14. Database Design (Core Tables)

♦ Tables

- users
- groups
- documents
- versions
- collaborators
- activities
- reminders
- tags

◆ Key Relationships

- Group → Many Documents
 - Group → Parent Group (self reference)
 - Document → Many Versions
 - Document → Many Collaborators
-

15. Important Backend Features

◆ Autosave

- Debounce updates
 - PATCH endpoint
-

◆ Soft Delete

status = "PENDING_DELETED"
deletedAt = timestamp

◆ Scheduler (Important)

Use Spring Scheduler:

@Scheduled(cron = "0 0 0 * * ?")

- Delete items older than 30 days
-

16. Security Layer

- JWT filter
- Role-based access
- Per-document permissions
- Rate limiting (optional)

- Input validation
-

17. Performance Optimization

- Pagination everywhere
- Lazy loading (JPA)
- Caching (Redis optional)
- Index DB columns